Software Development Skills Front-End, Online course

Learning diary

Lappeenranta-Lahti University of Technology LUT

Computational Engineering

Nguyen Minh Thuan Bui, 003043417

Software Development Skills Front-End, Online course

Spring - Summer 2026

LEARNING TOPICS

1. INTRODUCTION AND BASE HTML

Date: 24.4.2026
Activity: Watching video about structuring the project and implement it right away
Learning outcome: Structure of a webpage and how responsive design looks like

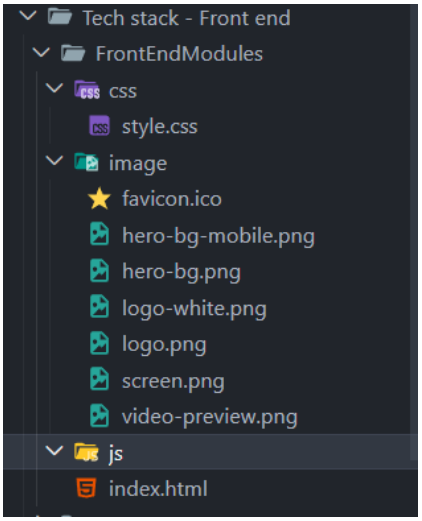
This introduction part is quick for me since I already learned designing Figma and do some simple html, CSS and JS back in high school. I got briefed about where the tutorial's Figma and GitHub repo used for the video at. The entire video will focus on mainly base page, and adding inner pages are purely optional. The video will go one by one of these orders: Navbar => Main area => Video section => Testimonials (lots of utility classes) => Pricing cards => FAQs => Footer. Also introduce how websites should response differently on desktops and mobiles, basically on different devices.

What I do is first I clone the git into my folder:

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

D:\LUTCodeFiles\Tech stack - Front end>git clone https://github.com/bradtraversy/saas-landing-page
Cloning into 'saas-landing-page'...
remote: Enumerating objects: 24, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 24 (delta 2), reused 1 (delta 1), pack-reused 16 (from 1)
Receiving objects: 100% (24/24), 450.88 KiB | 4.17 MiB/s, done.
Resolving deltas: 100% (4/4), done.
```

And then setting up the folders, assets and installing extensions such as Live Server and Prettier Code for further learnings:

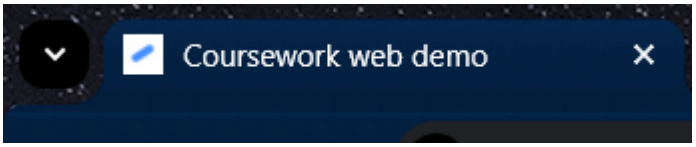


2. LINKS AND CORE CSS

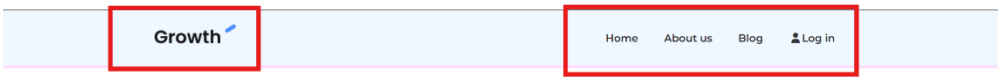
Date: 25.4.2026
Activity: Watch video about code structure of HTML and some basics CSS. Set up the HTML file and do a little bit of navbar
Learning outcome: How to initialize an HTML, how to link files, how to divide different containers into each divs and some basic CSS

For today, I fill up the HTML and CSS files with some contents. I set up the navbar in the HTML, basically make it exist first, and then I refine it with some CSS. Starting with the HTML file, I link the website with various references, such as Google fonts for getting the font, Font Awesome to get the icon, Style.css to link the HTML file with the CSS file, and finally create a title and a web icon for the webpage.

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>William Rover Web Project</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Montserrat:ital,wght@0,100..900;1,100..900&family=Mulish:ital,wght@0,200..1000;1,200..1000&di
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/7.0.1/css/all.min.css" integrity="sha512-2SwdPD6INVrV/lHTZbO2nodK
    <link rel="stylesheet" href="css/style.css">
    <link rel="shortcut icon" type="image/x-icon" href="image/favicon.ico">
</head>
```



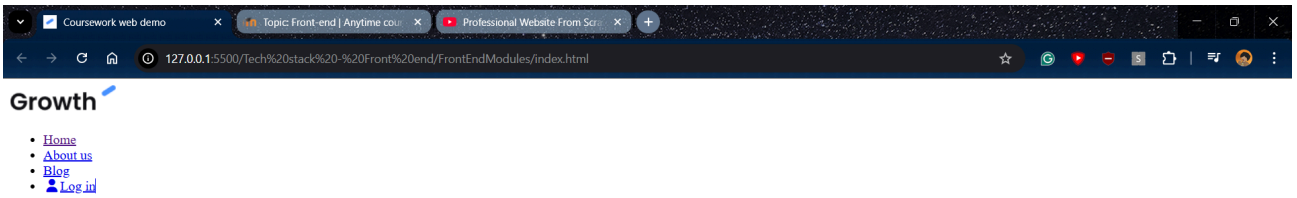
Next, I start to structure my navbar layout (a good layout will make it easier to modify later). Looking at the demo navbar, we can see it get divided into 2 smaller containers:



a logo container and a list container. For the logo, I can just put a <a> tag and an <img> tag, to basically it displays an image but also serves as a link. For the list of navigations, I used , which stands for unordered list, to automatically list all the elements. So when I done the HTML, this is the code structure and this is what it should look like (see next page)

```html
<nav class="navbar">
    <div class="container">
        <div class="logo">
            <a href="index.html"><img src="image/logo.png" alt="logo"></a>
        </div>

        <div class="main-menu">
            <ul>
                <li><a href="index.html">Home</a></li>
                <li><a href="#">About us</a></li>
                <li><a href="#">Blog</a></li>
                <li><a class="btn btn-light" href="#"><i class="fas fa-user"></i>Log in</a></li>
            </ul>
        </div>
    </div>
</nav>
```



It is currently looks ugly, and that is because there are no CSS yet.

So starting with the CSS file. We can see that in the web demo, the navbar is a margined by a little down and to the right. While it is not a problem right now, it will be once I add in more of my custom margins or paddings, so to avoid that, I will basically reset all the margin and padding of all elements (code on the left). Next, I want to use the font, so I have to declare it in the body. Also in the background, I can set the background color and probably add some distance between the lines, do I use line-height for that (code on the right). After that, I want to hide all the bullet points of the ul, so to do that I just put ul {list-style:none;}, and it will stay hidden. Next up, I do not want the list to order vertically but horizontally instead, so I will apply a flex display. Basically this will make the alignment flexible, hence the name. If I set the height low enough, it will

```css
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}
```

```css
body {
    font-family: 'Montserrat', sans-serif;
    font-size: 16px;
    line-height: 1.5;
    background: #fdf;
}
```

```css
.navbar .container {
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.navbar .main-menu ul {
    display: flex;
}
```

automatically rearrange the list to horizontally. (code on the left)

Then, I want to make the buttons that when I hover over, it will change color slowly at the duration of 0.5 seconds. To do that, I use the ::hover modifier for the very same element and at a transition line to the original element (code on the right).

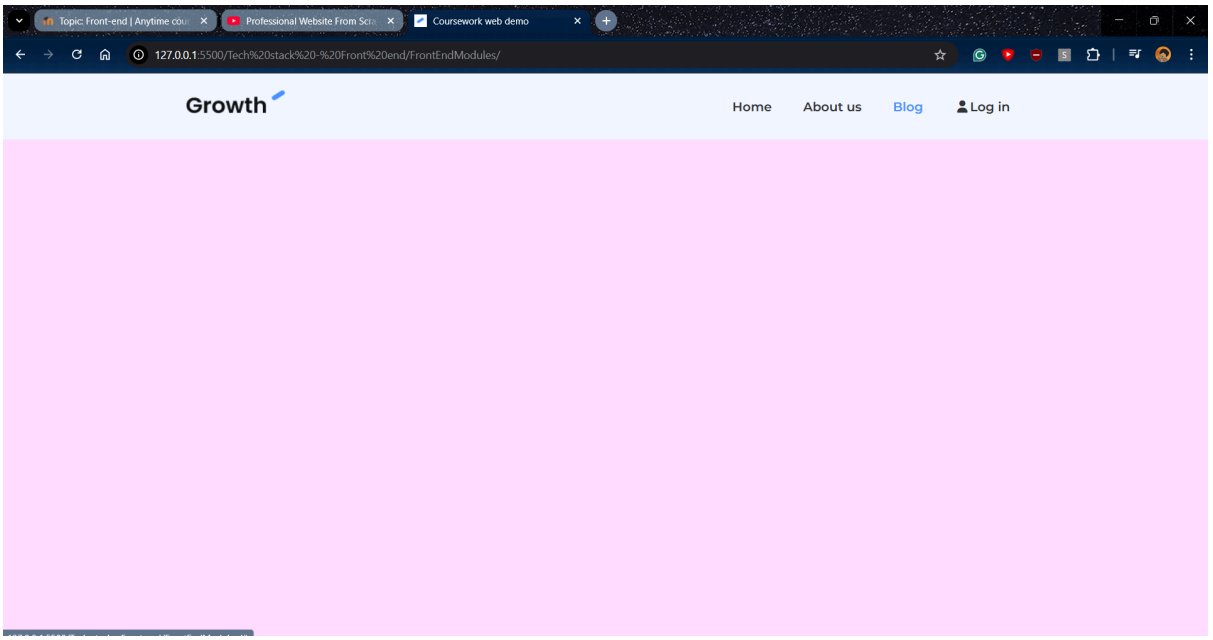
```css
.navbar ul li a {
    padding: 10px 20px;
    display: block;
    font-weight: 600;
    transition: 0.5s;
}

.navbar ul li a:hover {
    color: var(--primary-color);
}
```

Lastly, I used the var() inside. That is because I want it to be dynamic, meaning the color can be changed much later. To initiate variables for css, I use the element :root and delcare all the variables starting with “--"

```css
:root {
    --primary-color: #4891ff;
    --l-color: #f4f4f6;
    --dark-color: #111;
}
```

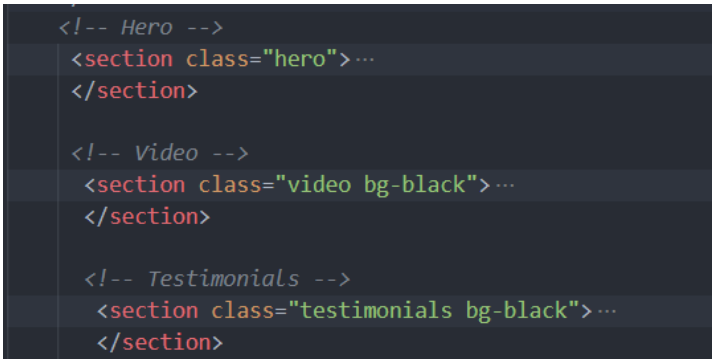
This is what the final result looks like:



3. BUTTONS AND UTILITY CLASS

Date: 28.4.2026
Activity: Watch video about code utilities and using class effectively. Set up Hero section and Video section
Learning outcome: Learn code reusability by using the same class on different divs/sections

This whole section is about code reusability. Mainly, three elements are being reused: button, background and text. I also got to know the <section> tag, and the difference between it and the tag is that the section tag "grouped" the content together, while the div just divide the content generally (although they are most likely the same thing in a big picture).



For the reuse part, the general concept is that I wrote the properties in the CSS file, and then if I want to use those properties, I will use the same class name for the HTML class as the one defined in the CSS. This is like a printer in real life: there can be different paper materials, but the print can remain the same. As one can see in the following code, most reused elements have the same properties, with the only difference is in the numerical value (see code next page)

```css
/* Buttons */
.btn {
    display: inline-block;
    padding: 14px 20px;
    background-color: var(--l-color);
    color: #333;
    font-weight: 600;
    text-decoration: none;
    border: none;
    border-radius: 10px;
    cursor: pointer;
    transition: 0.5s;
}

.btn:hover {
    opacity: 0.8;
    /* background-color: #d1d1d1; */
}

.btn-primary {
    background: var(--primary-color);
    color: #fff;
}

.btn-dark {
    background-color: var(--dark-color);
    color: #fff;
}

.btn-block {
    display: block;
    width:100%;
}
```

```css
/* Text */
.text-xxl {
    font-size: 3rem;
    line-height: 1.2;
    font-weight: 600;
    margin: 40px 0 20px;
}
.text-xl {
    font-size: 2.2rem;
    line-height: 1.4;
    font-weight: normal;
    margin: 40px 0 20px;
}

.text-lg {
    font-size: 1.8rem;
    line-height: 1.4;
    font-weight: normal;
    margin: 30px 0 20px;
}

.text-md {
    font-size: 1.2rem;
    line-height: 1.4;
    font-weight: normal;
    margin: 20px 0 10px;
}

.text-sm {
    font-size: 0.9rem;
    line-height: 1.4;
    font-weight: normal;
    margin: 10px 0 5px;
}

.text-center {
    text-align: center;
}
```

```css
/* Background */
.bg-primary {
    background: var(--primary-color);
    color: #fff;
}

.bg-light {
    background: var(--l-color);
    color: #333;
}
.bg-dark {
    background: var(--primary-color);
    color: #fff;
}
.bg-black {
    background: #000;
    color: #fff;
}
```

```html
<a href="#" class="btn btn-primary">Get started</a>
<a href="#" class="btn btn-primary"><i class="fas fa-laptop"></i> Book a demo</a>
```

For the Hero section, I learnt that instead of having to add another <img> tag and then align it to the bottom, I can just expand the container's height and then use the image as a background and put it to the bottom of the container. For the Video section, I found out another way to center a div is to use flex box, set the direction to column and align all the item to the center.

4. CSS GRIDS AND CARDS

Date: 04.05.2026
Activity: Watch video about using CSS grids and make repeating cards. Set up testimonials and pricing sections
Learning outcome: Knowing the difference between grids and flex box and how to set up cards

This section is quite similar to utility class section (e.g reusing code). Flexbox and grid are quite similar, the only difference is that grid will align based on predefined layout (layout before content), and flexbox will align based on the content (content before layout). Also got to learn a new unit fr, which is base on the fraction of the parent div/section, and to know another alternative for margin is gap, which will automatically space out each card.

```css
/* Testimonials */
.testimonials {
    padding: 40px 0;

}

.testimonials .testimonials-heading {
    width: 700px;
    margin-bottom: 40px;
}
.testimonials-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 30px;
}
/* Card */
.card {
    background: #fff;
    color: #000;
    border-radius: 10px;
    padding:20px;
}
.card p:nth-child(2) {
    margin-top: 30px;
    font-weight: bold;
}

/* Pricing */
.pricing-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    margin-top: 50px;
    gap:30px;
}
```

For the card section, I only need to define a single css property for 1 div. Then the only thing I need to do is to repeat those card div in html, and change the content if wanted.

```html
<div class="card">···
</div>
<div class="card">···
</div>

<div class="card">···
</div>
```

5. FAQ ELEMENTS

Date: 04.05.2026
Activity: Learnt to make each FAQ element closed upon clicking and finalize the footer
Learning outcome: Learnt to operate within addEventListenr in JS, use toggle() and querySelector

For the FAQ, I first make a list of FAQs like I have done with cards. Then I add an additional class element to the first FAQ, that is .open. Basically any FAQ that has the .open will display the content of the FAQ, otherwise the display will be set to none.

```
.faq .faq-group .faq-group-body {
    display: none;
}
.faq .faq-group .faq-group-body.open {
    display: block;
}
```

Additionally, I also got to write code so that only 1 FAQ element can be opened at the time. So when that one element is opened, the icon will be turned to plus and .open modifier is removed.

```
        icon.classList.toggle('fa-plus');
        icon.classList.toggle('fa-minus');

        groupBody.classList.toggle('open')
        const otherGroup = faqContainer.querySelectorAll('.faq-group')
        otherGroup.forEach((otherGroup) =>{
            if(otherGroup != group) {
                const otherGroupBody = otherGroup.querySelector('.faq-group-body')
                const otherIcon = otherGroup.querySelector('.faq-group-header i')

                otherGroupBody.classList.remove('open')
                otherIcon.classList.remove('fa-minus')
                otherIcon.classList.add('fa-plus')
```

For the footer, I again divide it into 4 divs, and then use grid to divide them, with the input form takes up 2/5 space of the footer.

```
<!-- Footer -->
<footer class="footer bg-black">
  <div class="container">
      <div class="footer-grid">
          <div>···
          </div>

          <div>···
          </div>

          <div>···
          </div>

          <div>···
          </div>
      </div>
  </div>
</footer>
```

```
.footer-grid {
    display: grid;
    grid-template-columns: 2fr 1fr 1fr 1fr;
    gap: 30px;
    justify-content: center;
    align-items: center;
}
```

6. MOBILE MENU AND RESPONSIVENESS

Date: 06.05.2026
Activity: Set up responsiveness for display screen of tablet and smaller
Learning outcome: Learnt to use @media css property to make website responsive to different screens

I first make the main media query (the laptop screen) to be hidden when the screen is about the size of a tablet.

```
@media (max-width: 670px) {
  .navbar .main-menu {
    display: none;
  }

  .navbar .hamburger-button {
    display: block;
  }
```

```
/* Hamburger Button */
.hamburger-button {
  display: none;
```

Additionally, I also got to do the hamburger menu, which only shows up when the screen is small enough also and add some extra css properties for it to work in mobile display.

7. WEBSITE DEPLOYMENT

Date: 06.05.2026
Activity: Deploy some past personal website using github pages
Learning outcome: Learnt to host website using github.io pages for free

For this activity, I go a little different path from the provided video (I prefer using GUI than terminals) because the process is much faster for me. I first push my code to github repositories. Then, I head to Pages section in Settings, click Deploy from a branch, then I just need to wait for it to get hosted. I managed to practice this by hosting one of my old project last year: https://williamrover.github.io/date-safedial/